NLP 243 – Homework 1

Relation Extraction from Natural Language with BoW + MLP

Jose Alvarez Maciel

October 18, 2025

1 Task & Dataset

In my understanding, this task is a multi-label classification problem where each sentence may have multiple labels. The given data is represented as a bag-of-words (BoW) vector, which uses 942 words. I think with the 942 words, there could be an imbalance with some labels that can occur frequently, for example, "entity/location". Another thing that can play a role is the sparsity, since this seems to have high-dimensional features but mostly holds zeros, and that's each containing only a few words. From looking at the documentation about BoW it appears to ignore word order, and it can't distinguish certain ones.

With this dataset, I can conclude that there is an imbalance across the 19 label categories, which requires using a weighted loss to handle rare classes. Since BoW features are sparse and high-dimensional, the model must learn to identify the patterns between word occurrences and target labels without relying on the sentence structure.

2 Model & Variations

The model that was used was a feed-forward (MLP), which is structured into three main types of layers. With the BoW, its vectors are inputs into a two-layer feed, and the first layer projects the 942-dimensional vectors into a smaller space that can be adjusted before training (156–256 units), which was followed by using ReLU to use its activation and dropout for regularization. The final layer then outputs 19 logits that correspond to each label.

During the task, I experimented with the hidden dimension, dropout rate, learning rate, and optimizer. I also increased the hidden size to see if I could improve the model's capacity, but it did increase the risk of overfitting. The higher drop rate that was used was 0.5, which did reduce overfitting slightly, but it did slow the convergence. One thing I noticed was that when I used the Adam optimizer, it outperformed the SGD by taking in the learning rates for sparse gradients. A moderate learning rate 1e-2 gave the best tradeoff for balance and speed.

3 Experiments & Findings

I ran nine experiments (A–K) by varying key hyperparameters such as hidden layer size, dropout rate, learning rate, optimizer, and regularization. Each of these configurations was trained for the same number of epochs and evaluated using weighted F1 across its training, validation, and test splits.

We trained a set of configurations by varying hidden size, dropout, learning rate, optimizer, and regularization. We report weighted F1 for train/validation/test.

#	Hidden	Dropout	LR	Batch	Opt.	Layers	Reg.	Train F1	Val F1	Test F1
A	256	0.3	1e-2	65	Adam	2	None	0.9970	0.8903	0.9022
B	156	0.3	1e-2	65	Adam	2	None	0.9933	**0.9156**	**0.9074**
C	156	0.5	1e-2	32	Adam	2	None	0.9847	0.8899	0.8983
D	156	0.5	1e-3	64	Adam	2	None	0.9183	0.8692	0.8637
E	256	0.5	5e-2	65	Adam	2	None	0.9443	0.8431	0.8556
G	256	0.3	5e-1	65	Adam	2	None	0.9550	0.8439	0.8446
H	256	0.3	5e-1	65	SGD	2	None	0.6340	0.6060	0.6086
I	256	0.5	1e-2	65	SGD	2	L2=1e-4	0.1743	0.1976	0.1754
K	128	0.0	1e-2	65	Adam	2	None	0.9688	0.8997	0.8765

Table 1: Hyperparameter sweep and results (weighted F1).

4 Training Curves (Required Plots)

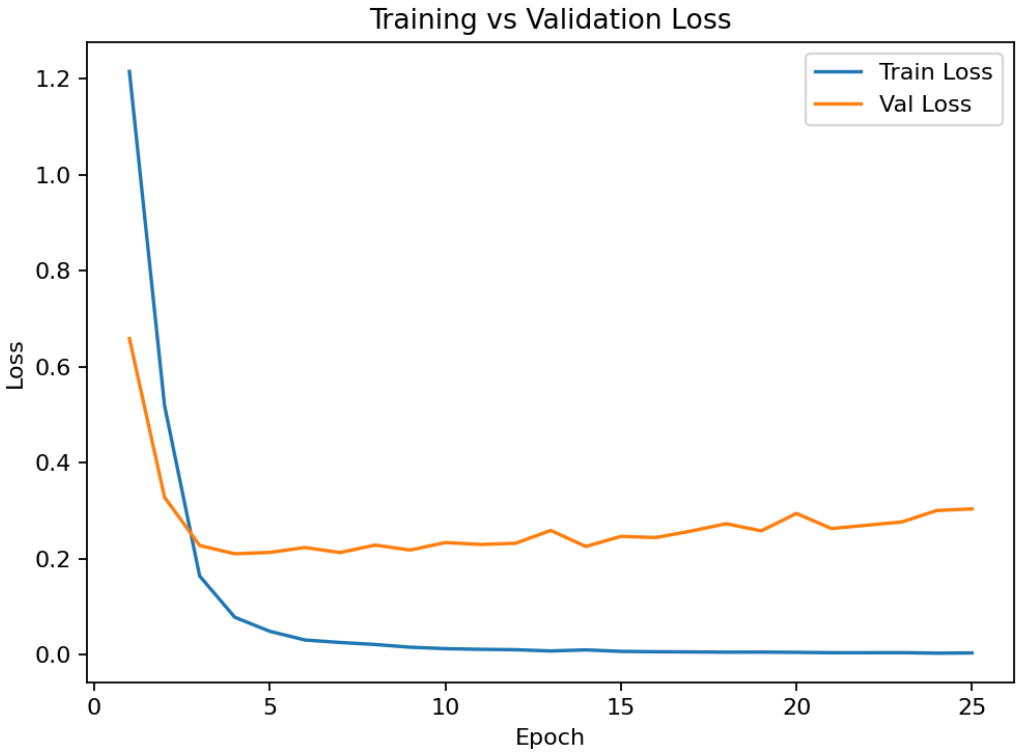

Figure 1: Training vs. Validation Loss (lower is better).

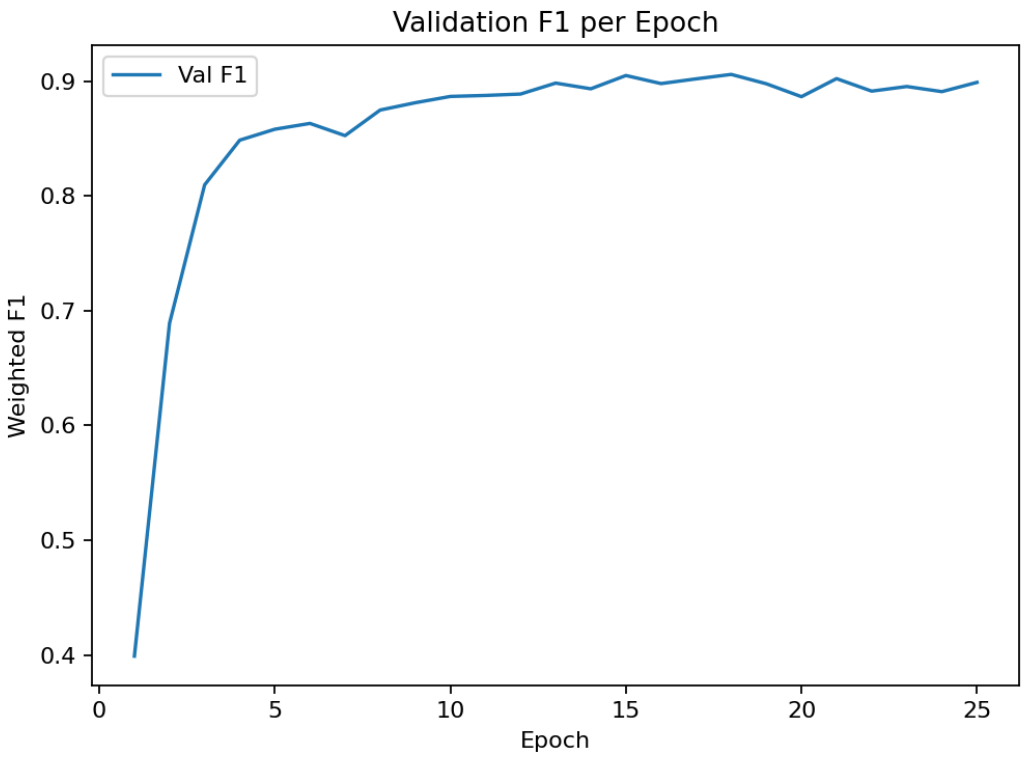

Figure 2: Validation weighted F1 per epoch (higher is better).

5 Conclusion

The current plot shows how the weighted F1 score changes across 25 training epochs.

The model starts with a low F1 (0.4), which means it's weak in the initial performance when weights are random. In the first 5–10 epochs, the F1 rapidly increases to around 0.9, which shows that the model is learning at a good pace, which is reflected by the learning rate of 1e-2 and using Adam. After the 10, it starts to curve and plateau, which means the model has likely converged.